

Labo 00 — Questions

Répondez sans relire la théorie. Une réponse d'une à cinq phrases suffit ; ce qui compte est le raisonnement, pas le vocabulaire.

Question 1 [Compréhension]

Un binaire compilé pour Linux (par exemple `nginx`) fonctionne à l'identique sur Ubuntu, Debian et Alpine, mais pas du tout sur Windows sans WSL. Expliquez ce que ce binaire attend de son système, et pourquoi la distribution n'a pas d'importance alors que le noyau en a.

Question 2 [Analyse]

Vous lancez `sleep 300 &` dans un terminal, puis vous fermez ce terminal. Un peu plus tard, `ps -ef` montre que le processus `sleep` existe toujours, mais que son PPID vaut désormais `1`. Que s'est-il passé, et pourquoi le système a-t-il besoin de ce mécanisme ?

Question 3 [Diagnostic]

Un collègue vous montre ceci :

```
$ ./deploy.sh
bash: ./deploy.sh: Permission denied
$ echo $?
126
```

Le fichier existe et il en est propriétaire. Donnez la cause exacte, la commande qui la confirme, la commande qui la corrige — et une seconde façon de lancer le script sans rien corriger du tout.

Question 4 [Prédiction]

Prédisez les deux lignes affichées par cette séquence, puis justifiez la différence :

```
MSG=bonjour
bash -c 'echo 1: $MSG'
export MSG
bash -c 'echo 2: $MSG'
```

Question 5 [Diagnostic]

Dans un script d'exploitation, vous trouvez `echo $?` qui affiche `137` juste après l'arrêt brutal d'un service Java. Décomposez ce nombre, dites ce qui est arrivé au processus, et pourquoi ce code précis est célèbre dans le monde des conteneurs.

Question 6 [Analyse]

Beaucoup d'administrateurs pressés font systématiquement `kill -9` au lieu de `kill`. Expliquez la différence de mécanisme entre les deux, ce que perd concrètement une application de type base de données dans le second cas, et le rapport avec la façon dont Docker arrête un conteneur (`docker stop`).

Question 7 [Diagnostic]

Observez :

```
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 652 mars 31 13:31 /etc/shadow
$ sudo cat /etc/shadow    # fonctionne
```

En vous appuyant sur la ligne du `ls -l`, expliquez précisément pourquoi le premier `cat` échoue et pourquoi le second réussit. Qui pourrait lire ce fichier sans `sudo` ?

Question 8 [Prédiction]

Que contient le fichier `resultat.txt` et qu'affiche l'écran après cette commande, sachant que `/date-inconnue` n'existe pas ?

```
ls /etc/hostname /date-inconnue > resultat.txt 2> erreurs.txt
```

Et que changerait `2>&1` placé après `> resultat.txt` ?

Question 9 [Analyse]

`ss -tlnp` sur un serveur montre ces deux lignes :

```
LISTEN 0  511      127.0.0.1:6379  0.0.0.0:*    users:(("redis-server",pid=812,fd=6))
LISTEN 0  511      0.0.0.0:8080   0.0.0.0:*    users:(("java",pid=944,fd=23))
```

Quelle est la différence de portée entre ces deux services ? Depuis un autre poste du réseau, lequel pouvez-vous joindre ? Pourquoi ce détail deviendra-t-il important quand vous publierez des ports de conteneurs ?

Question 10 [Compréhension]

Votre utilisateur (UID 1000) lance `python3 -m http.server 80` et obtient `PermissionError: [Errno 13] Permission denied`, alors que le port 8080 fonctionne. Expliquez la règle en jeu, sa raison d'être historique, et la conséquence directe pour Podman rootless.

Question 11 [Analyse]

`ls /proc` montre des centaines de répertoires, et pourtant `df -h` ne montre aucun espace disque consommé par `/proc` ; `findmnt -t proc` révèle un système de fichiers de type `proc`. Expliquez ce qu'est réellement `/proc`, d'où viennent ses « fichiers », et donnez un exemple d'information qu'on va y chercher.

Question 12 [Diagnostic]

Sur une machine fraîchement installée, un collègue a copié un outil dans `~/outils/monoutil`, vérifié avec `ls` qu'il est bien exécutable, mais obtient :

```
$ monoutil
bash: monoutil: command not found
$ echo $?
127
```

Expliquez comment le shell a cherché `monoutil`, pourquoi il ne l'a pas trouvé, et donnez deux façons durables (et une immédiate) de rendre la commande utilisable.

Question 13 [Analyse]

La méthodologie « 12-factor » impose de configurer une application par **variables d'environnement** plutôt que par des fichiers modifiés à la main. En vous appuyant sur ce que vous savez de l'héritage parent → enfant et du cycle de vie d'un processus, expliquez pourquoi ce choix convient parfaitement à des processus jetables et relançables — comme le seront vos conteneurs Spring Boot au labo 08.